Module Interface Specification for Squigglr

Team #17, Squigglr
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

November 13, 2026

1 Revision History

Date	Developer(s)	Change
11 November 2026	Ahmed Elzaria	Added the introduction, the notation, the types shared between modules and the module decomposition, and gave each module its own file (#324).
11 November 2026	Ahmed Elzaria	Added the MIS of the Audio Output, Lesson Catalogue and Feedback Modules, and the feedback, image and sound types (#325).
11 November 2026	Ahmed Elzaria	Added the MIS of the Attempt, Attempt Evaluation and Geometry Modules (#326).
11 November 2026	Ahmed Elzaria	Added the MIS of the Guided Lesson Control and Improvement Measure Modules, with the access program for each lesson transition of SRS Section 9.2 (#327).
11 November 2026	Ahmed Elzaria	Added the MIS of the Attempt Record Module, and checked the lesson and attempt modules against the MIS checklist (#328).
11 November 2026	Ahmed Elzaria	Added his generative AI use disclosure (#329).
11 November 2026	Muhammad Huzaifah	Added the MIS of the Adult Access, Learner Profile and Learner Settings Modules (#337).
11 November 2026	Muhammad Huzaifah	Added the MIS of the Progress Summary and Adult View Modules (#338).
11 November 2026	Muhammad Huzaifah	Added the MIS of the Storage, Connection Recovery and Audit Log Modules (#339).
11 November 2026	Muhammad Huzaifah	Checked his modules against the MIS and Writing checklists and added his generative AI use disclosure (#340).
13 November 2026	Ahmed Elzaria	Addressed Huzaifah's review of #330 to #335: who reads the adult-set values, a colour in FeedbackT, ObservedT shared, a clock in M6 and M4, a device check before `select`, the adult kept by M4, the stage chosen on accepting a suggestion, and missed parts noted only at the end of an attempt.

2 Symbols, Abbreviations and Acronyms

The symbols, abbreviations and acronyms are those of the SRS (Squigglr Team, 2026b) (Section 4, Naming Conventions and Terminology) and of the MG (Squigglr Team, 2026a) (Section 2, Reference Material). Symbolic constants, such as `STAGE_PASS` and `ATTEMPT_LIMIT`, are defined with their units, ranges and defaults in the Symbolic Constants subsection of the SRS. This document adds the following.

Symbol	Description
M	Module, numbered as in the MG (for example, M3)
MIS	Module Interface Specification
D, T, E, I	The stages of a guided lesson: demonstration, full-outline tracing, endpoint-only guidance and independent drawing (SRS Section 9.2)

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Introduction **1**

4 Notation **1**

4.1 Types Shared Between Modules . . . 2

5 Module Decomposition **4**

6 MIS of the Input Device Module (M1) **5**

7 MIS of the Display Module (M2) **6**

8 MIS of the Audio Output Module (M3) **7**

8.1 Module . . . 7
8.2 Uses . . . 7
8.3 Syntax . . . 7
8.3.1 Exported Constants . . . 7
8.3.2 Exported Access Programs . . . 7
8.4 Semantics . . . 7
8.4.1 State Variables . . . 7
8.4.2 Environment Variables . . . 7
8.4.3 Assumptions . . . 7
8.4.4 Access Routine Semantics . . . 8
8.4.5 Local Functions . . . 8

9 MIS of the Guided Lesson Control Module (M4) **9**

9.1 Module . . . 9
9.2 Uses . . . 9
9.3 Syntax . . . 9
9.3.1 Exported Constants . . . 9
9.3.2 Exported Types . . . 9
9.3.3 Exported Access Programs . . . 10
9.4 Semantics . . . 10
9.4.1 State Variables . . . 10
9.4.2 Environment Variables . . . 11
9.4.3 Assumptions . . . 11
9.4.4 Access Routine Semantics . . . 11
9.4.5 Local Functions . . . 13

10 MIS of the Lesson Catalogue Module (M5) 14
10.1 Module . . . 14
10.2 Uses . . . 14
10.3 Syntax . . . 14
10.3.1 Exported Constants . . . 14
10.3.2 Exported Types . . . 14
10.3.3 Exported Access Programs . . . 14
10.4 Semantics . . . 15
10.4.1 State Variables . . . 15
10.4.2 Environment Variables . . . 15
10.4.3 Assumptions . . . 15
10.4.4 Access Routine Semantics . . . 15
10.4.5 Local Functions . . . 15

11 MIS of the Attempt Module (M6) 16
11.1 Template Module . . . 16
11.2 Uses . . . 16
11.3 Syntax . . . 16
11.3.1 Exported Constants . . . 16
11.3.2 Exported Types . . . 16
11.3.3 Exported Access Programs . . . 16
11.4 Semantics . . . 17
11.4.1 State Variables . . . 17
11.4.2 Environment Variables . . . 17
11.4.3 Assumptions . . . 17
11.4.4 Access Routine Semantics . . . 17
11.4.5 Local Functions . . . 18

12 MIS of the Attempt Evaluation Module (M7) 19
12.1 Module . . . 19
12.2 Uses . . . 19
12.3 Syntax . . . 19
12.3.1 Exported Constants . . . 19
12.3.2 Exported Access Programs . . . 19
12.4 Semantics . . . 19
12.4.1 State Variables . . . 19
12.4.2 Environment Variables . . . 19
12.4.3 Assumptions . . . 19
12.4.4 Access Routine Semantics . . . 20
12.4.5 Local Functions . . . 20

13 MIS of the Feedback Module (M8) 21
13.1 Module . . . 21
13.2 Uses . . . 21
13.3 Syntax . . . 21
13.3.1 Exported Constants . . . 21
13.3.2 Exported Access Programs . . . 21
13.4 Semantics . . . 21
13.4.1 State Variables . . . 21
13.4.2 Environment Variables . . . 21
13.4.3 Assumptions . . . 21
13.4.4 Access Routine Semantics . . . 22
13.4.5 Local Functions . . . 22

14 MIS of the Improvement Measure Module (M9) 23
14.1 Module . . . 23
14.2 Uses . . . 23
14.3 Syntax . . . 23
14.3.1 Exported Constants . . . 23
14.3.2 Exported Access Programs . . . 23
14.4 Semantics . . . 23
14.4.1 State Variables . . . 23
14.4.2 Environment Variables . . . 23
14.4.3 Assumptions . . . 23
14.4.4 Access Routine Semantics . . . 23
14.4.5 Local Functions . . . 24

15 MIS of the Attempt Record Module (M10) 25
15.1 Module . . . 25
15.2 Uses . . . 25
15.3 Syntax . . . 25
15.3.1 Exported Constants . . . 25
15.3.2 Exported Types . . . 25
15.3.3 Exported Access Programs . . . 26
15.4 Semantics . . . 26
15.4.1 State Variables . . . 26
15.4.2 Environment Variables . . . 26
15.4.3 Assumptions . . . 26
15.4.4 Access Routine Semantics . . . 27
15.4.5 Local Functions . . . 27

16 MIS of the Creative Drawing Module (M11) 28

17 MIS of the Refinement Control Module (M12) 29

18 MIS of the Child Screen Module (M13) 30

19 MIS of the Adult Access Module (M14) 31
19.1 Module . . . 31
19.2 Uses . . . 31
19.3 Syntax . . . 31
19.3.1 Exported Constants . . . 31
19.3.2 Exported Types . . . 31
19.3.3 Exported Access Programs . . . 31
19.4 Semantics . . . 32
19.4.1 State Variables . . . 32
19.4.2 Environment Variables . . . 32
19.4.3 Assumptions . . . 32
19.4.4 Access Routine Semantics . . . 32
19.4.5 Local Functions . . . 33

20 MIS of the Learner Profile Module (M15) 34
20.1 Module . . . 34
20.2 Uses . . . 34
20.3 Syntax . . . 34
20.3.1 Exported Constants . . . 34
20.3.2 Exported Types . . . 34
20.3.3 Exported Access Programs . . . 34
20.4 Semantics . . . 34
20.4.1 State Variables . . . 34
20.4.2 Environment Variables . . . 35
20.4.3 Assumptions . . . 35
20.4.4 Access Routine Semantics . . . 35
20.4.5 Local Functions . . . 35

21 MIS of the Learner Settings Module (M16) 36
21.1 Module . . . 36
21.2 Uses . . . 36
21.3 Syntax . . . 36
21.3.1 Exported Constants . . . 36
21.3.2 Exported Types . . . 36
21.3.3 Exported Access Programs . . . 37
21.4 Semantics . . . 37
21.4.1 State Variables . . . 37
21.4.2 Environment Variables . . . 37
21.4.3 Assumptions . . . 37
21.4.4 Access Routine Semantics . . . 37
21.4.5 Local Functions . . . 38

22 MIS of the Progress Summary Module (M17) 39
22.1 Module . . . 39
22.2 Uses . . . 39
22.3 Syntax . . . 39
22.3.1 Exported Constants . . . 39
22.3.2 Exported Types . . . 39
22.3.3 Exported Access Programs . . . 40
22.4 Semantics . . . 40
22.4.1 State Variables . . . 40
22.4.2 Environment Variables . . . 40
22.4.3 Assumptions . . . 40
22.4.4 Access Routine Semantics . . . 40
22.4.5 Local Functions . . . 41

23 MIS of the Adult View Module (M18) 42
23.1 Module . . . 42
23.2 Uses . . . 42
23.3 Syntax . . . 42
23.3.1 Exported Constants . . . 42
23.3.2 Exported Types . . . 42
23.3.3 Exported Access Programs . . . 42
23.4 Semantics . . . 43
23.4.1 State Variables . . . 43
23.4.2 Environment Variables . . . 43
23.4.3 Assumptions . . . 43
23.4.4 Access Routine Semantics . . . 44
23.4.5 Local Functions . . . 44

24 MIS of the Stroke Module (M19) 45

25 MIS of the Geometry Module (M20) 46
25.1 Module . . . 46
25.2 Uses . . . 46
25.3 Syntax . . . 46
25.3.1 Exported Constants . . . 46
25.3.2 Exported Types . . . 46
25.3.3 Exported Access Programs . . . 46
25.4 Semantics . . . 46
25.4.1 State Variables . . . 46
25.4.2 Environment Variables . . . 47
25.4.3 Assumptions . . . 47
25.4.4 Access Routine Semantics . . . 47
25.4.5 Local Functions . . . 47

26 MIS of the Edit History Module (M21) **48**

27 MIS of the Refinement Engine Module (M22) **49**

28 MIS of the Storage Module (M23) **50**
28.1 Module . . . 50
28.2 Uses . . . 50
28.3 Syntax . . . 50
28.3.1 Exported Constants . . . 50
28.3.2 Exported Types . . . 50
28.3.3 Exported Access Programs . . . 51
28.4 Semantics . . . 51
28.4.1 State Variables . . . 51
28.4.2 Environment Variables . . . 51
28.4.3 Assumptions . . . 51
28.4.4 Access Routine Semantics . . . 51
28.4.5 Local Functions . . . 52

29 MIS of the Connection Recovery Module (M24) **53**
29.1 Module . . . 53
29.2 Uses . . . 53
29.3 Syntax . . . 53
29.3.1 Exported Constants . . . 53
29.3.2 Exported Types . . . 53
29.3.3 Exported Access Programs . . . 53
29.4 Semantics . . . 53
29.4.1 State Variables . . . 53
29.4.2 Environment Variables . . . 53
29.4.3 Assumptions . . . 54
29.4.4 Access Routine Semantics . . . 54
29.4.5 Local Functions . . . 54

30 MIS of the Audit Log Module (M25) **55**
30.1 Module . . . 55
30.2 Uses . . . 55
30.3 Syntax . . . 55
30.3.1 Exported Constants . . . 55
30.3.2 Exported Types . . . 55
30.3.3 Exported Access Programs . . . 55
30.4 Semantics . . . 55
30.4.1 State Variables . . . 55
30.4.2 Environment Variables . . . 55
30.4.3 Assumptions . . . 55

30.4.4 Access Routine Semantics 56
30.4.5 Local Functions 56

31 Appendix — Generative AI Use Disclosure **58**
31.1 AI Tools Used 58
31.2 How and Where Used 58
31.3 Verification of AI-Generated Content 58

32 Appendix — Reflection **60**

3 Introduction

The following document details the Module Interface Specifications (MIS) for Squigglr, a website on which a young child practises drawing lines, shapes and simple drawings with a stylus. In guided lessons the guidance is removed stage by stage, and in the creative space the child draws freely and can have a drawing refined. A supporting adult chooses the lessons, sets the values that control them and sees the child's progress.

Complementary documents include the System Requirement Specifications (SRS) (Squigglr Team, 2026b) and the Module Guide (MG) (Squigglr Team, 2026a). The MG names each module and the secret it hides; this document gives each module's interface. The full documentation and implementation can be found at https://github.com/AHMEDELZARIA/squigglr.

4 Notation

The structure of the MIS for modules comes from Hoffman and Strooper (1995), with the addition that template modules have been adapted from Ghezzi et al. (2003). The mathematical notation comes from Chapter 3 of Hoffman and Strooper (1995). For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | ... | c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Squigglr.

Data Type	Notation	Description
Boolean	$\mathbb{B}$	true or false
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
natural number with zero	$\mathbb{N}_0$	a number without a fractional component in [0, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)

The specification of Squigglr uses some derived data types: sequences, strings, sets, tuples and enumerations. Sequences are lists filled with elements of the same data type, written "seq of T". Strings are sequences of characters. Sets are unordered collections without repeats, written "set of T". Tuples contain a list of values, potentially of different types, and their fields are named, as in $p.x$. An enumeration is a type with a fixed list of values, written as the set of those values, such as $\{D, T, E, I\}$. In addition, Squigglr uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

The symbolic constants of the SRS keep their names, written in typewriter font, for example `STAGE_PASS`. A module whose behaviour depends on an adult-set value (the nine values of BR-008 in the SRS) is given the value as an input. Only the Guided Lesson Control Module (M4) reads these values from the Learner Settings Module (M16), when a lesson is chosen, and it passes them on to the modules it calls, so that those modules never read the Learner Settings Module themselves. A constant that the team fixes belongs to the one module that uses it. It is an exported constant of that module when its value is known, as `TRACE_TOL` is, and part of the module's secret while the SRS still leaves its value to be decided.

4.1 Types Shared Between Modules

Table 1 lists the types that appear in the interfaces of more than one module, so that every module uses the same names. They are plain data types, so using one does not make a module use another module. The last column names the module whose data the type describes and which decides its values. StrokeT is the exception: it is an abstract data type, so a module that calls its access programs uses the Stroke Module (M19). A module that only passes a stroke on, as the Guided Lesson Control Module (M4) does, does not. A type that one module defines and other modules only name is defined in that module's Exported Types, and the others refer to it there.

Table 1: Types shared between modules

Type	**Definition**	**Owner**
PointT	tuple of $(x : \mathbb{R}, y : \mathbb{R})$, a position on the drawing area	Notation
StrokeT	An abstract data type: the points of one stylus contact, in the order drawn, with the time of each point	M19
PathT	seq of PointT, a line through the points in order. The parts of a lesson's figure are paths.	Notation
DateTimeT	A date and time, in Coordinated Universal Time	Notation
StageT	$\{D, T, E, I\}$, the stages of a guided lesson (the set L of SRS Section 9.2)	M5
DrawingStageT	$\{T, E, I\}$, the stages at which the child draws (the set A of SRS Section 9.2)	M5
LessonIdT	Identifies one lesson in the catalogue	M5
DifficultyT	tuple of $(kind : \{missedPart, missedEndpoint, offFigure\}, index : \mathbb{N}_0)$, one predefined difficulty (BR-007): a required part or endpoint of the figure that was not drawn, given by its index in the figure, or strokes drawn away from the figure (index 0)	M5

Type	Definition	Owner
FigureT	tuple of (*parts* : seq of PathT, *endpoints* : seq of PointT), the required parts and endpoints of a lesson's figure (SRS Table **??**)	M5
GuideT	tuple of (*outline* : seq of PathT, *endpoints* : seq of PointT), what is shown to guide the child at a drawing stage	M5
ClassT	{*complete*, *incomplete*, *invalid*} (the set R of SRS Section 9.2)	M7
GuidanceT	{*assisted*, *independent*} (the set M of SRS Section 9.2)	M7
EvaluationT	tuple of (*class* : ClassT, *guidance* : GuidanceT, *result* : $\mathbb{R}$, *passed* : $\mathbb{B}$, *difficulties* : set of DifficultyT), the outcome of evaluating the strokes of an attempt. The result is on the 0–100 scale of the SRS.	M7
ObservedT	tuple of (*difficulty* : DifficultyT, *corrected* : $\mathbb{B}$), a predefined difficulty seen during an attempt and whether the child corrected it later in the same attempt (FR-REC-003)	M6
FeedbackKindT	{*stagePassed*, *lessonComplete*, *keepTrying*, *notEvaluable*, *checkIn*, *lessonEnded*}, the kinds of message the child can be given during a guided lesson, other than the feedback on a predefined difficulty	M8
FeedbackT	tuple of (*message* : string, *image* : ImageIdT, *colour* : ColourIdT, *sound* : SoundIdT), the content of one message to the child. The colour is the one used to show the message and to mark where a missed part or endpoint is	M8
ColourIdT	Identifies one colour that Squigglr uses for feedback	M8
ImageIdT	Identifies one image that Squigglr shows the child	M8
SoundIdT	Identifies one sound that Squigglr can play	M8
AdultIdT	Identifies a signed-in supporting adult	M14
LearnerIdT	Identifies one learner profile	M15
SettingsT	tuple of the nine adult-set values of one learner (BR-008), each field named after its SRS constant: *stagePass*, *difficultyStreak*, *attemptLimit*, *improveWindow*, *attemptExtension*, *attemptLimitMax*, *attemptIdle*, *lessonIdlePrompt* and *lessonIdleEnd*	M16

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project (Squiglr Team, 2026a). The module numbers are the same in both documents. The last column names the issue in which each module's interface is written.

Table 2: Module Hierarchy

Level 1	**No.**	**Level 2**	**MIS part**
Hardware-Hiding	M1	Input Device Module	#265
	M2	Display Module	#265
	M3	Audio Output Module	#264
Behaviour-Hiding	M4	Guided Lesson Control Module	#264
	M5	Lesson Catalogue Module	#264
	M6	Attempt Module	#264
	M7	Attempt Evaluation Module	#264
	M8	Feedback Module	#264
	M9	Improvement Measure Module	#264
	M10	Attempt Record Module	#264
	M11	Creative Drawing Module	#265
	M12	Refinement Control Module	#265
	M13	Child Screen Module	#265
	M14	Adult Access Module	#266
	M15	Learner Profile Module	#266
	M16	Learner Settings Module	#266
	M17	Progress Summary Module	#266
	M18	Adult View Module	#266
Software Decision	M19	Stroke Module	#265
	M20	Geometry Module	#264
	M21	Edit History Module	#265
	M22	Refinement Engine Module	#265
	M23	Storage Module	#266
	M24	Connection Recovery Module	#266
	M25	Audit Log Module	#266

Each module's section below follows the same layout: Module, Uses, Syntax (exported constants, exported types and access programs), and Semantics (state variables, environment variables, assumptions, access routine semantics and local functions). In Design Revision -1 the access routine semantics are left for Revision 0. The Uses of each module are those of the uses hierarchy in the MG.

6 MIS of the Input Device Module (M1)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

7 MIS of the Display Module (M2)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

8 MIS of the Audio Output Module (M3)

8.1 Module

AudioOutput

8.2 Uses

None.

8.3 Syntax

8.3.1 Exported Constants

None.

8.3.2 Exported Access Programs

Name	In	Out	Exceptions
play	s: SoundIdT	–	–
stop	–	–	–
setMuted	m: $\mathbb{B}$	–	–
isMuted	–	$\mathbb{B}$	–

8.4 Semantics

8.4.1 State Variables

- *muted*: $\mathbb{B}$, whether sound is turned off for the active learner, initially false.
- *playing*: $\mathbb{B}$, whether a sound is playing, initially false.

8.4.2 Environment Variables

- *speaker*: SoundIdT $\cup$ $\{\perp\}$, the sound the browser's audio output on the device is playing, or $\perp$ when it plays none.

8.4.3 Assumptions

- The caller (the Child Screen Module, M13) calls `setMuted` with the negation of the active learner's audio setting (audio on means not muted) from the Learner Settings Module (M16) before it plays any sound for that learner, and again whenever the active learner or the setting changes (NFR-UX-010).

- Every sound is also shown on screen as a message, so a sound that the browser does not play loses no information (NFR-UX-021).

8.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief: `play` plays *s* on the speaker unless *muted*, stopping any sound already playing; `stop` stops the sound that is playing, so that a long sound can be stopped from the screen it plays on (NFR-UX-022); `setMuted` sets *muted*, and muting also stops the sound that is playing; `isMuted` gives *muted*.

8.4.5 Local Functions

None.

9 MIS of the Guided Lesson Control Module (M4)

9.1 Module

LessonControl

9.2 Uses

Input Device Module (M1), Lesson Catalogue Module (M5), Attempt Module (M6), Improvement Measure Module (M9), Attempt Record Module (M10), Adult Access Module (M14), Learner Settings Module (M16).

9.3 Syntax

9.3.1 Exported Constants

None.

9.3.2 Exported Types

- StatusT = {*noLesson*, *active*, *suggested*, *ended*}, the session status q of SRS Section 9.2, with *noLesson* before the first lesson is chosen.
- ReportT = tuple of (*feedback* : seq of FeedbackKindT, *difficulties* : set of DifficultyT), what the child is told after an event, in the order it is given. The difficulties are those of the attempt that just ended.

9.3.3 Exported Access Programs

Name	In	Out	Exceptions
deviceSupported	–	$\mathbb{B}$	–
select	a: AdultIdT, p: LearnerIdT, ℓ: LessonIdT, l: StageT	–	adultCheckFailed, unknownLesson, notOwner, unavailable
beginPractice	–	–	notAtDemonstration
addStroke	s: StrokeT	ReportT	noAttempt
tick	–	ReportT	–
answerSuggestion	a: AdultIdT, *accept*: $\mathbb{B}$, l: DrawingStageT	–	adultCheckFailed, noSuggestion, notEarlierStage
stop	–	–	–
status	–	StatusT	–
lesson	–	LessonIdT	noLesson
stage	–	StageT	noLesson
furthest	–	StageT	noLesson
figure	–	FigureT	noLesson
demonstration	–	seq of PathT	noLesson
guide	–	GuideT	noLesson, notAtDrawingStage

9.4 Semantics

9.4.1 State Variables

The state follows the state σ of SRS Section 9.2:

- a: AdultIdT, the adult who chose the lesson, given to every call to the Attempt Record Module (M10).
- ℓ: LessonIdT, p: LearnerIdT and l: StageT, the lesson, the learner and the current stage.
- q: StatusT, initially *noLesson*.
- n_f, n_a, k: $\mathbb{N}_0$, the failed attempts, the evaluable attempts and the current failure streak at this stage.
- H: seq of $\mathbb{R}$, the stage results of the evaluable attempts at this stage, in order.
- s: $\mathbb{B}$, whether a go-back suggestion is pending, and i: $\mathbb{N}_0$, its index in the lesson record.

- z: $\mathbb{B}$ and x: $\mathbb{N}_0$, whether the extension has been granted at this stage, and the evaluable attempts made since.
- f: StageT, the furthest stage reached in the lesson.
- *set*: SettingsT, the learner's adult-set values when the lesson was chosen.
- *att*: AttemptT, the current attempt, if there is one.
- r: LessonRecordIdT, the lesson record of this run of the lesson.
- *prompted*: $\mathbb{B}$, whether the idle check-in is showing.

9.4.2 Environment Variables

- *clock*: DateTimeT, the current time.

The time of the child's last input is read from the Input Device Module (M1).

9.4.3 Assumptions

- The Child Screen Module (M13) passes each finished stroke to `addStroke`, calls `tick` about once a second while a lesson is showing, and shows each ReportT with the Feedback Module (M8).
- The Adult View Module (M18) calls `stop` before the adult switches learner or ends the session. Before `select`, it calls `deviceSupported`, and when that gives false it shows the adult a notice that results may be unreliable and calls `select` only if the adult goes on (SAF-RAY-002, UC-002).
- One lesson runs at a time on a device.

9.4.4 Access Routine Semantics

The full semantics are given in Revision 0, as a transition for each row of the SRS table of guided lesson state transitions (SRS Table **??**). In brief:

- `deviceSupported` gives the result of the device check of the Input Device Module (SAF-RAY-002).
- `select` runs the adult check of the Adult Access Module for *lessonChoice*, ends any lesson in progress as unfinished, reads the learner's settings, starts a new lesson record, and sets a, p and (ℓ, l) with all stage counts, the history, the suggestion and the extension reset (AdultSelect; FR-LRN-016, FR-LRN-018, SAF-RAY-001). It raises `notOwner` if p is not one of a's learners, and `unavailable` if the learner's settings cannot be read.

- `beginPractice` moves from stage D to stage T and starts the first attempt.
- `addStroke` gives the stroke to the current attempt. An attempt ends at a stylus lift only when it passes (FR-LRN-039). Then `addStroke` saves the attempt record, updates the counts, and moves to the next stage or completes the lesson (FR-LRN-023). The report gives the feedback for what happened.
- `tick` lets an open attempt end as not passed or invalid after `ATTEMPT_IDLE` (FR-LRN-024). It then saves the attempt record and updates the counts, except for an attempt that ends as invalid, which is neither saved nor counted (FR-LRN-010, FR-LRN-032). After a counted attempt, it applies the first of these that holds, in this order: the end-at-limit rule ends the lesson (FR-LRN-021, SAF-RAY-005); the extension rule grants the one extension (FR-LRN-020, SAF-RAY-006); a go-back suggestion is offered if SuggestBack holds (FR-LRN-019). Unless the lesson ends or the stage changes, a new attempt then starts at the same stage (FR-LRN-015). `tick` also shows the check-in after `LESSON_IDLE_PROMPT` with no input, and ends the lesson after `LESSON_IDLE_END` (SAF-RAY-009). The report gives the feedback for what happened.
- `answerSuggestion` runs the adult check for *suggestionAnswer*. Declining clears s and resets k, and l is not used; accepting moves to the earlier stage l. Both are saved in the lesson record (FR-REC-010). A suggestion that is never answered stays pending and blocks nothing.
- `stop` interrupts the current attempt and ends the lesson as unfinished (FR-REC-005, FR-REC-007).
- The other programs give the state, and the figure, demonstration and guide of the current lesson and stage from the Lesson Catalogue.

A pending suggestion that a pass, `select` or the end of the lesson clears is saved as ignored (FR-REC-010), before `endLesson` of the Attempt Record Module is called.

Table 3 shows the access program that carries out each transition of the SRS table, so that no transition is left without one.

Table 3: SRS lesson transitions and the access programs of M4

SRS transition	**Access program**
Adult selects lesson / starting level	`select`
Begin practice after demonstration	`beginPractice`
Finalize evaluable attempt	`addStroke` (a pass at a stylus lift) or `tick` (after `ATTEMPT_IDLE`)
Interrupted or invalid input	`tick` (an invalid attempt) or `stop` (an interruption)
Offer suggestion	`tick`, after the attempt ends at `ATTEMPT_IDLE`

SRS transition	Access program
Adult declines suggestion	`answerSuggestion` with *accept* = *false*
Suggestion not answered	None needed: the suggestion stays pending until `addStroke`, `tick`, `select` or `stop` clears it
Adult accepts suggestion	`answerSuggestion` with *accept* = *true*
End at limit	`tick`, after the attempt ends at `ATTEMPT_IDLE`
Extend at base limit	`tick`, after the attempt ends at `ATTEMPT_IDLE`
Passing a nonfinal drawing stage	`addStroke`
Passing the final drawing stage	`addStroke`

In every state, $n_a \leq$ `ATTEMPT_LIMIT_MAX`, $0 \leq x \leq$ `ATTEMPT_EXTENSION`, and $z = \mathit{false} \Rightarrow x = 0$ (invariant I5 of SRS Section 9.2).

9.4.5 Local Functions

These are defined in Revision 0, from SRS Section 9.2:

- *suggestBack*: $\mathbb{B}$, the predicate SuggestBack.
- *atLimit*: $\mathbb{B}$, the guard of the end-at-limit rule.
- *canExtend*: $\mathbb{B}$, the guard of the extension rule.
- *next*: $\{T, E\} \to \{E, I\}$, the next stage.

10 MIS of the Lesson Catalogue Module (M5)

10.1 Module

LessonCatalogue

10.2 Uses

Storage Module (M23).

10.3 Syntax

10.3.1 Exported Constants

None. The lessons are data, read at run time, so that a lesson can be added, changed or removed without a change to the code (NFR-MAINT-001, LC-001).

10.3.2 Exported Types

- CategoryT = {*line*, *shape*, *simpleDrawing*}, the categories of the approved catalogue (FR-LRN-001).

10.3.3 Exported Access Programs

Name	In	Out	Exceptions
load	–	–	catalogueUnavailable, badLesson
lessons	–	seq of LessonIdT	notLoaded
name	ℓ: LessonIdT	string	notLoaded, unknownLesson
category	ℓ: LessonIdT	CategoryT	notLoaded, unknownLesson
figure	ℓ: LessonIdT	FigureT	notLoaded, unknownLesson
demonstration	ℓ: LessonIdT	seq of PathT	notLoaded, unknownLesson
guide	ℓ: LessonIdT, l: DrawingStageT	GuideT	notLoaded, unknownLesson
difficulties	ℓ: LessonIdT, l: DrawingStageT	set of DifficultyT	notLoaded, unknownLesson

10.4 Semantics

10.4.1 State Variables

- *loaded*: $\mathbb{B}$, whether the catalogue has been read, initially false.
- *catalogue*: seq of LessonT, the approved lessons in the order they are offered, where LessonT is the local type below.

LessonT = tuple of (*id* : LessonIdT, *name* : string, *category* : CategoryT, *figure* : FigureT, *demonstration* : seq of PathT, *difficulties* : DrawingStageT $\rightarrow$ set of DifficultyT).

10.4.2 Environment Variables

None. The stored lessons are reached through the Storage Module (M23).

10.4.3 Assumptions

- `load` is called once, before any other access program, when Squigglr starts.
- The team checks a lesson before it is stored. The checks in `load` only stop a damaged or incomplete lesson from being offered.

10.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief: `load` reads the stored lessons into *catalogue*. When the Storage Module (M23) raises `unavailable` while the lessons are read, `load` raises `catalogueUnavailable`. It raises `badLesson` if a lesson has no figure, an empty part or a difficulty whose index is not in its figure; `lessons` gives the identifiers in *catalogue* in order; `name`, `category` and `figure` give those fields of one lesson; `demonstration` gives the parts of the figure in the order the example draws them (FR-LRN-003); `guide` gives the whole figure as the outline at stage T, only the endpoints at stage E, and nothing at stage I (FR-LRN-004 to FR-LRN-006); `difficulties` gives the predefined difficulties of that stage of the lesson. Each raises `notLoaded` before `load` and `unknownLesson` for an identifier not in *catalogue*.

10.4.5 Local Functions

None.

11 MIS of the Attempt Module (M6)

11.1 Template Module

Attempt

11.2 Uses

Input Device Module (M1), Attempt Evaluation Module (M7), Stroke Module (M19).

11.3 Syntax

11.3.1 Exported Constants

None.

11.3.2 Exported Types

- AttemptT =?, one attempt at one drawing stage of a lesson.
- OutcomeT = {*open*, *passed*, *notPassed*, *invalid*, *interrupted*}, how the attempt ended, or *open* while it has not ended.

11.3.3 Exported Access Programs

Name	In	Out	Exceptions
new AttemptT	ℓ: LessonIdT, l: DrawingStageT, f: FigureT, ds: set of DifficultyT, *stagePass*: $\mathbb{R}$, *attemptIdle*: $\mathbb{R}$	AttemptT	–
addStroke	s: StrokeT	–	attemptEnded
checkIdle	–	–	–
interrupt	–	–	attemptEnded
outcome	–	OutcomeT	–
evaluation	–	EvaluationT	notEvaluated
strokes	–	seq of StrokeT	–
observed	–	set of ObservedT	–
lesson	–	LessonIdT	–
stage	–	DrawingStageT	–
endedAt	–	DateTimeT	attemptOpen

11.4 Semantics

11.4.1 State Variables

- ℓ: LessonIdT and l: DrawingStageT, the lesson and stage of the attempt.
- f: FigureT and ds: set of DifficultyT, the lesson's figure and the predefined difficulties of the stage.
- *stagePass*: $\mathbb{R}$ and *attemptIdle*: $\mathbb{R}$, the learner's `STAGE_PASS` and `ATTEMPT_IDLE` when the attempt started.
- X: seq of StrokeT, the strokes drawn so far, in order.
- *last*: EvaluationT, the evaluation after the most recent stylus lift.
- *seen*: set of ObservedT, every difficulty seen so far.
- *out*: OutcomeT.
- t_{end}: DateTimeT, when the attempt ended.

11.4.2 Environment Variables

- *clock*: DateTimeT, the current time.

The time of the child's last input is read from the Input Device Module (M1).

11.4.3 Assumptions

- The Guided Lesson Control Module (M4) creates an attempt with values from the Lesson Catalogue (M5) and the Learner Settings Module (M16), passes each finished stroke to `addStroke` at the stylus lift that ends it, and calls `checkIdle` often enough that the attempt ends within a second of `ATTEMPT_IDLE`.
- The input device has already dropped duplicate reports of the same input event and points it cannot map to the drawing area, and keeps strokes the child draws separately, even where they overlap (SAF-AHM-001, SAF-AHM-002).

11.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `new AttemptT` starts an open attempt with no strokes.

- `addStroke` appends s to X, evaluates all of X with the Attempt Evaluation Module (FR-LRN-008, FR-LRN-038), and updates *seen*, marking a difficulty corrected when it is no longer observed. A difficulty about something the child has drawn, such as a part drawn off the figure, is noted at every stylus lift. A missed part or missed endpoint is noted only from the evaluation with which the attempt ends, because at an early stylus lift the child may simply not have drawn it yet (FR-REC-003). If the evaluation passes, the attempt ends as *passed* (FR-LRN-039).
- `checkIdle` ends an open attempt that has at least one stroke when `ATTEMPT_IDLE` has passed between the last input and *clock*: as *invalid* if the last evaluation is invalid (FR-LRN-010), and otherwise as *notPassed*. An attempt with no strokes is left open.
- `interrupt` ends the attempt as *interrupted*, keeping its strokes and difficulties and no evaluation result (FR-LRN-009, FR-REC-005).
- When the attempt ends, t_{end} is set from *clock*.
- The other programs give the state. `evaluation` raises `notEvaluated` before the first stroke and after the attempt ends as *interrupted*, and `endedAt` raises `attemptOpen` while the attempt is open. `addStroke` and `interrupt` raise `attemptEnded` when the attempt has already ended.

11.4.5 Local Functions

None.

12 MIS of the Attempt Evaluation Module (M7)

12.1 Module

AttemptEvaluation

12.2 Uses

Stroke Module (M19), Geometry Module (M20).

12.3 Syntax

12.3.1 Exported Constants

- `TRACE_TOL` = 5, in percent of the figure's size (the diagonal of its bounding box). This is the value proposed in the SRS; the team confirms it in PH-1, before the first evaluation test.
- `PART_COVER` = 90, in percent. Proposed in the SRS and confirmed with `TRACE_TOL`.

12.3.2 Exported Access Programs

Name	In	Out	Exceptions
evaluate	X: seq of StrokeT, f: FigureT, l: DrawingStageT, ds: set of DifficultyT, $stagePass$: $\mathbb{R}$	EvaluationT	emptyFigure

12.4 Semantics

12.4.1 State Variables

None.

12.4.2 Environment Variables

None.

12.4.3 Assumptions

- $stagePass$ is in the range of `STAGE_PASS`, 70 to 90, because the Learner Settings Module (M16) does not save a value outside it.
- ds holds only difficulties whose index is a part or endpoint of f, because the Lesson Catalogue (M5) does not offer a lesson that breaks this.

12.4.4 Access Routine Semantics

`evaluate`$(X, f, l, ds, stagePass)$: the full semantics are given in Revision 0, following SRS Section 9.2. In brief:

- output: $out := (C, G(l), v, C = complete \wedge v \geq stagePass, obs)$, where C is the class of X against f (*invalid* if X cannot be interpreted, *complete* if every required part has at least `PART_COVER` of its length within `TRACE_TOL` of a stroke and every required endpoint has a drawn point within `TRACE_TOL` of it, and *incomplete* otherwise), $G(l)$ is *assisted* at T and E and *independent* at I (FR-LRN-012), v is the stage result $100 \cdot \min(drawn, onFigure)$, and obs is the set of difficulties in ds that the strokes show (FR-LRN-014). At stage I, the strokes are first fitted to the figure's bounding box with `fitToBox` of the Geometry Module (M20): moved and uniformly scaled, as in SRS Section 9.2. The result is the same every time for the same inputs (NFR-PERF-004) and matches the definition in SRS Section 9.2 (NFR-PERF-010).

- exception: $exc := (f.parts = \langle\rangle \Rightarrow \text{emptyFigure})$.

12.4.5 Local Functions

These are defined in Revision 0, from SRS Section 9.2:

- *valid*: seq of StrokeT $\to \mathbb{B}$

- *complete*: seq of PathT $\times$ FigureT $\to \mathbb{B}$

- *drawn*: seq of PathT $\times$ FigureT $\to \mathbb{R}$, the share of the figure's length within `TRACE_TOL` of a stroke

- *onFigure*: seq of PathT $\times$ FigureT $\to \mathbb{R}$, the share of the drawn length within `TRACE_TOL` of the figure

- *tol*: FigureT $\to \mathbb{R}$, `TRACE_TOL` as a distance on the drawing area for this figure

13 MIS of the Feedback Module (M8)

13.1 Module

Feedback

13.2 Uses

None.

13.3 Syntax

13.3.1 Exported Constants

None.

13.3.2 Exported Access Programs

Name	In	Out	Exceptions
feedback	k: FeedbackKindT	FeedbackT	–
difficultyFeedback	d: DifficultyT	FeedbackT	–
allFeedback	–	set of FeedbackT	–

13.4 Semantics

13.4.1 State Variables

None.

13.4.2 Environment Variables

None.

13.4.3 Assumptions

- The Guided Lesson Control Module (M4) decides which kind of feedback is given after an attempt. This module decides only what that feedback says, shows and plays, so the wording, images, colours and sounds can change without a change to the rules that pick them (LC-005).
- On an attempt that does not pass, the caller gives the *keepTrying* feedback, which acknowledges the child's effort, before the feedback on any difficulty (NFR-UX-016).

- The Child Screen Module (M13) shows the message and image on screen and plays the sound through the Audio Output Module (M3), and shows where a missed part or endpoint is without drawing it (SAF-MOU-001).

13.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief: `feedback` gives the approved message, image, colour and sound for a kind of feedback; `difficultyFeedback` gives the approved feedback for the kind of a predefined difficulty (BR-007, FR-LRN-014); `allFeedback` gives every approved message, so that the wording can be checked against NFR-UX-004 and NFR-UX-015. Every message describes the drawing and not the child (FR-LRN-022), and no image or sound marks the drawing as wrong (NFR-UX-004). All three are total, so no exception is needed.

13.4.5 Local Functions

None.

14 MIS of the Improvement Measure Module (M9)

14.1 Module

ImprovementMeasure

14.2 Uses

None.

14.3 Syntax

14.3.1 Exported Constants

- `IMPROVE_MARGIN` $= 5$, in points out of 100, the smallest rise in the average stage result that counts as improvement. The team fixed this value (SRS Symbolic Constants).

14.3.2 Exported Access Programs

Name	In	Out	Exceptions
improving	H: seq of $\mathbb{R}$, w: $\mathbb{N}$	$\mathbb{B}$	–

14.4 Semantics

14.4.1 State Variables

None.

14.4.2 Environment Variables

None.

14.4.3 Assumptions

- w is the learner's `IMPROVE_WINDOW`, which the Learner Settings Module (M16) keeps between 1 and 3, and each value in H is a stage result between 0 and 100.

14.4.4 Access Routine Semantics

`improving`(H, w):

- output: $out := Improving(H, w)$, the predicate of SRS Section 9.2. It is false when H has fewer than $2w$ results. Otherwise it is true when the average of the last w results is at least `IMPROVE_MARGIN` above the average of the w results before them (FR-LRN-019, FR-LRN-020).

- exception: none.

The measure is the secret of this module (LC-008), so a different measure of improvement changes only this module.

14.4.5 Local Functions

None.

15 MIS of the Attempt Record Module (M10)

15.1 Module

AttemptRecord

15.2 Uses

Stroke Module (M19), Storage Module (M23), Connection Recovery Module (M24).

15.3 Syntax

15.3.1 Exported Constants

None.

15.3.2 Exported Types

- LessonRecordIdT, which identifies one lesson record.
- RecordOutcomeT = {*passed*, *notPassed*, *interrupted*}.
- EndingT = {*inProgress*, *completed*, *endedAtLimit*, *endedAtMax*, *endedIdle*, *unfinished*}, how a lesson ended (the lesson ending of the SRS data dictionary, with SAF-RAY-009's ending for inactivity).
- ResponseT = {*pending*, *accepted*, *declined*, *ignored*}, the adult's response to a go-back suggestion.
- SaveT = {*saved*, *pending*}, whether the server has confirmed a save, or the record is kept on the device until it does.
- AttemptRecordT = tuple of (*lesson* : LessonIdT, *stage* : DrawingStageT, *outcome* : RecordOutcomeT, *result* : $\mathbb{R}$, *difficulties* : set of ObservedT, *settings* : SettingsT, *strokes* : seq of StrokeT, *endedAt* : DateTimeT), one attempt (FR-REC-001 to FR-REC-005). An interrupted attempt has no result, so its *result* is not used.
- SuggestionT = tuple of (*from* : DrawingStageT, *offeredAt* : DateTimeT, *response* : ResponseT, *stage* : DrawingStageT $\cup \{\perp\}$), where *from* is the stage at which the failed attempts were made and *stage* is the earlier stage the adult chose on acceptance, or $\perp$ if the suggestion was not accepted (FR-REC-010).
- LessonRecordT = tuple of (*id* : LessonRecordIdT, *learner* : LearnerIdT, *lesson* : LessonIdT, *start* : StageT, *furthest* : StageT, *passes* : DrawingStageT $\to \mathbb{N}_0$, *extensions* : $\mathbb{N}_0$, *ending* : EndingT, *suggestions* : seq of SuggestionT, *startedAt* : DateTimeT), one start of a lesson (FR-REC-006, FR-REC-007, FR-REC-010).

15.3.3 Exported Access Programs

Name	In	Out	Exceptions
startLesson	a: AdultIdT, p: LearnerIdT, ℓ: LessonIdT, l: StageT	LessonRecord-IdT	notOwner
saveAttempt	a: AdultIdT, r: LessonRecordIdT, x: AttemptRecordT	SaveT	unknownRecord, lessonEnded
recordExtension	a: AdultIdT, r: LessonRecordIdT	SaveT	unknownRecord, lessonEnded
offerSuggestion	a: AdultIdT, r: LessonRecordIdT, l: DrawingStageT	$\mathbb{N}_0$	unknownRecord, lessonEnded
answerSuggestion	a: AdultIdT, r: LessonRecordIdT, i: $\mathbb{N}_0$, *resp*: ResponseT, l: DrawingStageT $\cup$ $\{\perp\}$	SaveT	unknownRecord, unknownSuggestion, alreadyAnswered, lessonEnded
endLesson	a: AdultIdT, r: LessonRecordIdT, e: EndingT, f: StageT	SaveT	unknownRecord, lessonEnded
lessonRecords	a: AdultIdT, p: LearnerIdT	seq of LessonRecordT	notOwner, unavailable
attempts	a: AdultIdT, r: LessonRecordIdT	seq of At-temptRecordT	unknownRecord, unavailable

15.4 Semantics

15.4.1 State Variables

None. The saved records are kept by the Storage Module (M23), and the records the server has not yet confirmed are kept on the device by the Connection Recovery Module (M24).

15.4.2 Environment Variables

None.

15.4.3 Assumptions

- The Guided Lesson Control Module (M4) is the only module that writes records, and the Progress Summary Module (M17) is the only module that reads them. The Adult View Module (M18) gets records from the Progress Summary, and gets the records not yet saved from the Connection Recovery Module.

- Who may delete records is still an open issue in the SRS (OI-02), so deletion is left to the Storage Module and is not part of this interface yet.

15.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `startLesson` starts a new lesson record for learner p, with ending *inProgress*, even when the same lesson was started before, so earlier attempts are kept (SAF-RAY-006). It raises `notOwner` if p is not one of a's learners.

- `saveAttempt` saves x under the lesson record r, and so to the learner of r only (SAF-RAY-007, FR-REC-008). Each attempt is saved exactly once, even when the Connection Recovery Module sends it again after the connection is lost. The output is *pending* while the record is kept on the device (SAF-RAY-003).

- `recordExtension`, `offerSuggestion` and `answerSuggestion` add an extension, a suggestion and the adult's response to the lesson record (SAF-RAY-006, FR-REC-010). `offerSuggestion` gives the index of the new suggestion, offered at stage l. `answerSuggestion` keeps l as the chosen stage when $resp = accepted$, and $\perp$ otherwise.

- `endLesson` sets the ending and the furthest stage. After it, the lesson record and its attempts never change (FR-REC-009), so every program that writes to the record raises `lessonEnded`.

- `lessonRecords` and `attempts` read back a learner's records, oldest first.

Records hold only guided practice, never a creative drawing (FR-CRT-028), keep each attempt's stage so that attempts with and without a guide stay apart (SAF-MOU-002), and hold no label or judgement of the child (SAF-RAY-004).

15.4.5 Local Functions

None.

16 MIS of the Creative Drawing Module (M11)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

17 MIS of the Refinement Control Module (M12)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

18 MIS of the Child Screen Module (M13)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

19 MIS of the Adult Access Module (M14)

19.1 Module

AdultAccess

19.2 Uses

Audit Log Module (M25).

19.3 Syntax

19.3.1 Exported Constants

None. `IDLE_TIMEOUT` and the form of the adult check are this module's secret.

19.3.2 Exported Types

- CredentialsT = tuple of (*email* : string, *password* : string), what an adult enters to sign up or sign in.
- SignUpResultT = {*created*, *emailInUse*, *notAccepted*}, where *notAccepted* means the credentials do not meet the rules of the sign-in service.
- AdultActionT = {*lessonChoice*, *suggestionAnswer*, *learnerSwitch*, *settingsChange*, *drawingRestore*}, the actions a child must not take alone: choosing a lesson and its starting stage and answering a go-back suggestion (SAF-RAY-001, FR-LRN-018), switching the active learner (SAF-RAY-001), changing a learner's settings (SAF-RAY-008) and restoring a drawing (FR-CRT-019).

19.3.3 Exported Access Programs

Name	In	Out	Exceptions
init	–	–	–
signUp	*c*: CredentialsT	SignUpResultT	unavailable
signIn	*c*: CredentialsT	$\mathbb{B}$	unavailable
signOut	–	–	notSignedIn
isSignedIn	–	$\mathbb{B}$	–
adult	–	AdultIdT	notSignedIn
adultCheck	*act*: AdultActionT	$\mathbb{B}$	notSignedIn

19.4 Semantics

19.4.1 State Variables

- *lastInput*: DateTimeT, the time of the last input to the page.

19.4.2 Environment Variables

- *auth*: tuple of (*accounts* : set of tuple of (*id* : AdultIdT, *c* : CredentialsT), *session* : AdultIdT $\cup$ $\{\perp\}$), the accounts of the hosted sign-in service and the session of this browser (NFR-OPS-002).
- *screen*: seq of seq of PixelT, the pixels of the browser window, where PixelT is the colour of one pixel. This module shows only the adult check on it.
- *input*: seq of InputEventT, the key presses and pointer events on the page, each with its time. It is used to time inactivity (SAF-SHA-006) and to read the answer to the adult check.
- *clock*: DateTimeT, the current time.

19.4.3 Assumptions

- `init` is called once, when the page is opened, before any other access program.
- A wrong password or an email already in use is the adult's mistake, not the caller's, so it is an output (false, or *emailInUse*), not an exception. Callers use `isSignedIn` before the programs that raise `notSignedIn`.

19.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `init` sets *lastInput* to the current time. A session kept by the browser from an earlier visit stays open.
- `signUp` creates an account with the sign-in service and signs the adult in. It collects nothing but the email and password (NFR-SEC-004).
- `signIn` opens a session and gives true, or gives false and logs a failed sign-in with the Audit Log Module (NFR-SEC-005). `signOut` ends the session, so it cannot be used again (NFR-SEC-002).
- `isSignedIn` is true while a session is open. A session with no input for `IDLE_TIMEOUT` is ended and the timeout is logged (SAF-SHA-006, NFR-SEC-005).
- `adult` gives the signed-in adult. Every module that reads or saves an adult's data is given this value by its caller (SAF-SHA-001).

- `adultCheck` shows a short check that a young child cannot pass alone and gives whether it was passed. When it gives false, the caller does not carry out *act* (SAF-RAY-001, SAF-RAY-008).

`unavailable` is raised when the sign-in service does not answer.

19.4.5 Local Functions

None.

20 MIS of the Learner Profile Module (M15)

20.1 Module

LearnerProfile

20.2 Uses

Storage Module (M23).

20.3 Syntax

20.3.1 Exported Constants

None.

20.3.2 Exported Types

- AvatarT, which identifies one of the avatar pictures that Squigglr offers. A profile never holds a photo or the name of the child (BR-001, NFR-SEC-004).
- LearnerT = tuple of (*id* : LearnerIdT, *avatar* : AvatarT), one learner profile (the learner profile of the SRS data dictionary).
- ActiveT = tuple of (*adult* : AdultIdT, *learner* : LearnerT), the active learner on this device and the adult who chose it.

20.3.3 Exported Access Programs

Name	In	Out	Exceptions
init	–	–	–
avatars	–	seq of AvatarT	–
create	*a*: AdultIdT, *av*: AvatarT	LearnerIdT	unavailable
learners	*a*: AdultIdT	seq of LearnerT	unavailable
setActive	*a*: AdultIdT, *p*: LearnerIdT	–	notOwner, unavailable
hasActive	–	$\mathbb{B}$	–
active	–	ActiveT	noActiveLearner
clearActive	–	–	–

20.4 Semantics

20.4.1 State Variables

- *current*: ActiveT $\cup$ $\{\perp\}$, the active learner on this device, or $\perp$ when there is none.

20.4.2 Environment Variables

None. The profiles are kept through the Storage Module (M23).

20.4.3 Assumptions

- `init` is called once, when the page is opened, before any other access program.
- a is the adult that the Adult Access Module (M14) gives as signed in.
- Before `setActive`, the caller has had the adult pass the adult check for *learnerSwitch* (SAF-RAY-001). Only the Adult View Module (M18) calls `create`, `setActive` and `clearActive`.
- Callers use `hasActive` before `active`.

20.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `init` sets *current* to $\perp$, so no learner is active until an adult chooses one.
- `avatars` gives the avatars an adult can choose from.
- `create` saves a new profile with an identifier that Squigglr generates, and gives that identifier (BR-001). It does not change *current*.
- `learners` gives a's profiles in the order they were created, and only those (NFR-SEC-001).
- `setActive` makes p the active learner, so that every attempt and drawing after it is saved to p only (SAF-RAY-007). It raises `notOwner` if p is not one of a's learners.
- `hasActive` tells whether there is an active learner, `active` gives it, and `clearActive` sets *current* to $\perp$, as the Adult View does when the adult signs out.

20.4.5 Local Functions

None.

21 MIS of the Learner Settings Module (M16)

21.1 Module

LearnerSettings

21.2 Uses

Storage Module (M23).

21.3 Syntax

21.3.1 Exported Constants

None. The range and default of each adult-set value are this module's secret; `range` gives them. Their values are those of the symbolic constants of the SRS (BR-008, BR-009).

21.3.2 Exported Types

- SettingNameT = {*stagePass*, *difficultyStreak*, *attemptLimit*, *improveWindow*, *attemptExtension*, *attemptLimitMax*, *attemptIdle*, *lessonIdlePrompt*, *lessonIdleEnd*}, the names of the fields of SettingsT (Table 1).
- RangeT = tuple of ($min : \mathbb{N}_0$, $max : \mathbb{N}_0$, $default : \mathbb{N}_0$), the team-set range of one adult-set value and its default, in the unit of its SRS constant: points out of 100 for *stagePass*, seconds for the three idle times, and attempts for the others.
- OptionsT = tuple of ($audio : \mathbb{B}$, $refinement : \mathbb{B}$), the audio setting (NFR-UX-010) and the refinement setting (FR-CRT-030) of one learner. The SRS data dictionary also lists accessibility settings, which the SRS has not defined yet, so they are not part of this interface.

21.3.3 Exported Access Programs

Name	In	Out	Exceptions
settings	a: AdultIdT, p: LearnerIdT	SettingsT	notOwner, unavailable
range	n: SettingNameT	RangeT	–
check	s: SettingsT	set of SettingNameT	–
save	a: AdultIdT, p: LearnerIdT, s: SettingsT	–	invalidSettings, notOwner, unavailable
options	a: AdultIdT, p: LearnerIdT	OptionsT	notOwner, unavailable
setAudio	a: AdultIdT, p: LearnerIdT, on: $\mathbb{B}$	–	notOwner, unavailable
setRefinement	a: AdultIdT, p: LearnerIdT, on: $\mathbb{B}$	–	notOwner, unavailable

21.4 Semantics

21.4.1 State Variables

None. The saved settings are kept through the Storage Module (M23).

21.4.2 Environment Variables

None.

21.4.3 Assumptions

- a is the adult that the Adult Access Module (M14) gives as signed in.
- Before `setAudio`, and before a `save` that changes values already saved for p, the caller has had the adult pass the adult check for *settingsChange* (SAF-RAY-008). The first `save` for a new profile needs no adult check, because it changes no saved value and the SRS has no adult check when a profile is created (use case UC-001). `setRefinement` needs no adult check, because the child may turn refinement off and on in the creative space (FR-CRT-030).
- The team-set ranges keep `LESSON_IDLE_END` below `IDLE_TIMEOUT` for every allowed value, so this module does not check that rule.

21.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `settings` gives p's saved values, with the default for each value the adult has not chosen (the middle of its range). `options` gives p's audio and refinement settings.

- `range` gives the range and default of value n.
- `check` gives the values of s that are outside their range or break a rule between values: `ATTEMPT_LIMIT` $>$ `DIFFICULTY_STREAK`, `ATTEMPT_LIMIT_MAX` $\geq$ `ATTEMPT_LIMIT`, and `ATTEMPT_IDLE` $<$ `LESSON_IDLE_PROMPT` $<$ `LESSON_IDLE_END`. Both values of a broken rule are given. An empty set means s can be saved.
- `save` saves s for p and raises `invalidSettings` if `check`$(s) \neq \emptyset$, so no value outside its range is ever saved (SAF-RAY-008). The values a lesson uses are those read when it is chosen.
- `setAudio` and `setRefinement` save one field of p's options.

Each program raises `notOwner` if p is not one of a's learners.

21.4.5 Local Functions

None.

22 MIS of the Progress Summary Module (M17)

22.1 Module

ProgressSummary

22.2 Uses

Attempt Record Module (M10).

22.3 Syntax

22.3.1 Exported Constants

- `PRACTICE_ONLY`: string = "This summary describes practice only. It is not an assessment or a diagnosis.", the practice-only statement that the Adult View Module (M18) shows on every summary (SAF-RAY-004).

22.3.2 Exported Types

The types LessonRecordT, AttemptRecordT and EndingT are those of the Attempt Record Module (M10).

- ViewChoiceT = tuple of (*lessons* : set of LessonIdT, *from* : DateTimeT $\cup \{\perp\}$, *to* : DateTimeT $\cup \{\perp\}$), the lessons and dates an adult asks a summary to cover (FR-PRG-007). The empty set covers every lesson, and $\perp$ sets no limit on the dates. Which recorded values are shown is chosen in the Adult View Module (M18).

- StageStatusT = $\{$*notAttempted*, *attempted*$\}$ (FR-PRG-010).

- StageSummaryT = tuple of (*stage* : DrawingStageT, *status* : StageStatusT, *passed* : seq of AttemptRecordT, *notPassed* : seq of AttemptRecordT, *interrupted* : seq of AttemptRecordT), the attempts at one drawing stage of one run of a lesson. Stages T and E are drawn with a guide and stage I without one (FR-PRG-003, FR-PRG-009).

- RunT = tuple of (*record* : LessonRecordT, *stages* : seq of StageSummaryT), one start of a lesson, with its starting stage, furthest stage, passes, extensions and ending (FR-PRG-002).

- LessonSummaryT = tuple of (*lesson* : LessonIdT, *runs* : seq of RunT, *limitEndings* : $\mathbb{N}_0$, *extensions* : $\mathbb{N}_0$), every run of one lesson in the order they started, with the number of runs that ended at a limit and the number of extensions granted (SAF-RAY-006).

- RecurringT = tuple of (*lesson* : LessonIdT, *stage* : DrawingStageT, *difficulty* : DifficultyT, *seen* : seq of AttemptRecordT, *corrected* : seq of AttemptRecordT), one recurring difficulty (FR-PRG-004).

- SummaryT = tuple of (*learner* : LearnerIdT, *lessons* : seq of LessonSummaryT, *recurring* : seq of RecurringT, *latest* : DateTimeT∪{⊥}), the progress summary of one learner (FR-PRG-001). It has no field that combines results across stages or lessons (FR-PRG-012).

22.3.3 Exported Access Programs

Name	In	Out	Exceptions
summary	a: AdultIdT, p: LearnerIdT, v: ViewChoiceT	SummaryT	notOwner, unavailable

22.4 Semantics

22.4.1 State Variables

None. A summary is built from the records each time it is asked for, and has no data of its own.

22.4.2 Environment Variables

None.

22.4.3 Assumptions

- a is the adult that the Adult Access Module (M14) gives as signed in.

22.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief, `summary` reads p's lesson records and their attempts with `lessonRecords` and `attempts` of the Attempt Record Module, keeps those of the lessons and dates in v, and builds the summary from them:

- Each lesson's runs are kept in the order they started, so no earlier attempt is hidden by a later run (SAF-RAY-006). A run's limit ending is an ending of *endedAtLimit* or *endedAtMax*.

- Each run has a stage summary for every drawing stage, so attempts with and without a guide are never added together (FR-PRG-003, SAF-MOU-002), interrupted attempts are kept apart from the others (FR-PRG-009), and a stage the child did not reach is *notAttempted* (FR-PRG-010).

- A difficulty is recurring when it was seen in at least `RECUR_MIN` of the learner's attempts at the same stage of the same lesson. *corrected* holds the attempts in which the child corrected it (FR-PRG-004). `RECUR_MIN` is this module's secret, proposed at 3 in the SRS.
- *latest* is the end time of the learner's latest attempt, including one saved just before the summary was asked for (FR-PRG-011), or $\perp$ if there is none. A learner with no records gets a summary with no lessons (FR-PRG-008).
- Every figure in the summary holds the attempt records behind it, so that the adult can open them (FR-PRG-005). Each record keeps the adult-set values it was made with (SAF-RAY-008).

`summary` raises `notOwner` if p is not one of a's learners, and `unavailable` if the records cannot be read.

22.4.5 Local Functions

None.

23 MIS of the Adult View Module (M18)

23.1 Module

AdultView

23.2 Uses

Display Module (M2), Guided Lesson Control Module (M4), Lesson Catalogue Module (M5), Refinement Control Module (M12), Adult Access Module (M14), Learner Profile Module (M15), Learner Settings Module (M16), Progress Summary Module (M17), Connection Recovery Module (M24).

23.3 Syntax

23.3.1 Exported Constants

None.

23.3.2 Exported Types

The inputs below use CredentialsT of the Adult Access Module, AvatarT of the Learner Profile Module, ViewChoiceT of the Progress Summary Module, and AttemptRecordT of the Attempt Record Module (M10), which this module only gets from the Progress Summary Module and passes on. DrawingIdT identifies one saved creative drawing and is the type of the Refinement Control Module.

- ViewT = {*signIn*, *profiles*, *settings*, *lessonChoice*, *progress*, *attempts*, *drawings*, *unsaved*}, the adult views.
- SummaryPartT = {*startingStage*, *furthestStage*, *passes*, *lessonEnding*, *limitCounts*, *recurring*, *interrupted*, *notAttempted*}, the recorded values an adult can choose to show in a progress summary (FR-PRG-007).

23.3.3 Exported Access Programs

There is one access program for each action an adult takes in the adult views. The inputs are what the adult enters or selects in the view.

Name	In	Out	Exceptions
init	–	–	–
signUp	c: CredentialsT	–	–
signIn	c: CredentialsT	–	–
signOut	–	–	–
addLearner	av: AvatarT, s: SettingsT	–	–
switchLearner	p: LearnerIdT	–	–
saveSettings	p: LearnerIdT, s: SettingsT	–	–
setAudio	p: LearnerIdT, on: $\mathbb{B}$	–	–
chooseLesson	ℓ: LessonIdT, l: StageT	–	–
acceptSuggestion	l: DrawingStageT	–	–
declineSuggestion	–	–	–
showProgress	p: LearnerIdT, v: ViewChoiceT, ps: set of SummaryPartT	–	–
showAttempts	rs: seq of AttemptRecordT	–	–
showDrawings	p: LearnerIdT	–	–
restoreDrawing	d: DrawingIdT	–	–
showUnsaved	–	–	–

23.4 Semantics

23.4.1 State Variables

- *view*: ViewT, the view shown.
- *parts*: set of SummaryPartT, the recorded values shown in a summary.

23.4.2 Environment Variables

- *screen*: seq of seq of PixelT, the pixels of the browser window while an adult view is shown, where PixelT is the colour of one pixel.
- *input*: seq of InputEventT, the adult's key presses and pointer events in the adult views, each with its time.

23.4.3 Assumptions

- `init` is called each time an adult view is opened, before any other access program.
- `showAttempts` is given the attempt records behind a figure of the summary on the screen, which the adult has just selected.

23.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `init` sets *view* to *profiles* if an adult is signed in and to *signIn* if not, and sets *parts* to every recorded value.
- Each program carries out one adult action and shows its result. A problem, such as a wrong password, a setting out of range, an unsupported device (SAF-RAY-002) or a server that does not answer, is shown on the screen in plain words (NFR-UX-017), so no program raises an exception.
- `switchLearner`, `saveSettings`, `setAudio` and `restoreDrawing` first run the adult check of the Adult Access Module, and change nothing if it is not passed (SAF-RAY-001, SAF-RAY-008, FR-CRT-019). The lesson choice and the answers to a go-back suggestion are passed to `select` and `answerSuggestion` of the Guided Lesson Control Module, which run the check themselves.
- `switchLearner` and `signOut` first call `stop` of the Guided Lesson Control Module. `signOut` also clears the active learner.
- `addLearner` creates the profile and saves its settings, so a new adult can set up a learner in one view (NFR-UX-009). It runs no adult check, because it saves the first values of a new profile and changes no saved value (SAF-RAY-008). `saveSettings` shows the values that `check` of the Learner Settings Module gives, and saves only when there are none.
- The lesson choice view lists the lessons of the Lesson Catalogue Module by name. Before it passes a lesson choice to `select`, it calls `deviceSupported` of the Guided Lesson Control Module. When that gives false, it shows the adult a notice that results may be unreliable and calls `select` only if the adult goes on (SAF-RAY-002, UC-002).
- `showProgress` shows the summary of p for v with the values in ps, and always with `PRACTICE_ONLY` (SAF-RAY-004). `showAttempts` shows every value of each record in rs, with its strokes drawn by the Display Module (FR-PRG-005, FR-PRG-006).
- `showDrawings` lists p's saved drawings that can be restored.
- `showUnsaved` lists the attempts and drawings that are still kept on the device and not yet saved (SAF-RAY-003, SAF-SHA-003).
- The adult views look different from the child's screens (NFR-UX-003), every action can be done with the keyboard alone (NFR-UX-023), and their text can be enlarged (NFR-UX-024).

23.4.5 Local Functions

None.

24 MIS of the Stroke Module (M19)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

25 MIS of the Geometry Module (M20)

25.1 Module

Geometry

25.2 Uses

Stroke Module (M19).

25.3 Syntax

25.3.1 Exported Constants

None.

25.3.2 Exported Types

- BoxT = tuple of ($minX : \mathbb{R}, minY : \mathbb{R}, maxX : \mathbb{R}, maxY : \mathbb{R}$), the smallest rectangle, with sides parallel to the drawing area, that holds a set of paths.

25.3.3 Exported Access Programs

Name	In	Out	Exceptions
toPath	s: StrokeT	PathT	–
length	P: PathT	$\mathbb{R}$	–
distance	p: PointT, Q: seq of PathT	$\mathbb{R}$	emptyPath
lengthWithin	P: PathT, Q: seq of PathT, d: $\mathbb{R}$	$\mathbb{R}$	emptyPath, negativeDistance
boundingBox	Q: seq of PathT	BoxT	emptyPath
diagonal	b: BoxT	$\mathbb{R}$	–
fitToBox	Q: seq of PathT, b: BoxT	seq of PathT	emptyPath
smooth	P: PathT	PathT	–
simplify	P: PathT, d: $\mathbb{R}$	PathT	negativeDistance

25.4 Semantics

25.4.1 State Variables

None.

25.4.2 Environment Variables

None.

25.4.3 Assumptions

None. Every distance is in the units of the drawing area, and no access program depends on the drawing library, so the library can be replaced without changing this interface.

25.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `toPath` gives the points of a stroke, in order, as a path.
- `length` gives the length of a path, and 0 for a path with fewer than two points.
- `distance` gives the smallest distance from p to any of the paths in Q.
- `lengthWithin` gives the length of the parts of P that are within distance d of a path in Q. The Attempt Evaluation Module uses it for how much of a required part is drawn and how much of a stroke lies on the figure.
- `boundingBox` and `diagonal` give the bounding box of Q and the length of its diagonal, which sets the size of a figure for `TRACE_TOL`.
- `fitToBox` moves and uniformly scales Q so that its bounding box is centred on b and its longer side matches b's (SRS Section 9.2, stage I).
- `smooth` and `simplify` give a smoother path and a path with fewer points that stays within d of P, for the Refinement Control Module (M12).

Each raises `emptyPath` when Q has no point, and `negativeDistance` when $d < 0$.

25.4.5 Local Functions

None.

26 MIS of the Edit History Module (M21)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

27 MIS of the Refinement Engine Module (M22)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

28 MIS of the Storage Module (M23)

28.1 Module

Storage

28.2 Uses

Audit Log Module (M25).

28.3 Syntax

28.3.1 Exported Constants

None.

28.3.2 Exported Types

- KindT = {*learner*, *settings*, *lesson*, *lessonRecord*, *attemptRecord*, *drawing*}, the kinds of saved item. Each kind is laid out by the module that owns it: the Learner Profile (M15), Learner Settings (M16), Lesson Catalogue (M5), Attempt Record (M10) and Creative Drawing (M11) Modules.
- ItemIdT, which identifies one saved item. No two items have the same identifier.
- ItemT, the data of one item, as its owner module lays it out: a tuple whose fields are numbers, Booleans, strings, date-times, or sequences and tuples of these. This module does not look inside an item, except to check its text before saving (NFR-SEC-006).
- EntryT = tuple of (*id* : ItemIdT, *learner* : LearnerIdT $\cup \{\perp\}$, *item* : ItemT), one saved item with the learner it belongs to.

28.3.3 Exported Access Programs

Name	In	Out	Exceptions
newId	–	ItemIdT	–
put	a: AdultIdT, k: KindT, p: LearnerIdT $\cup \{\perp\}$, id: ItemIdT, x: ItemT	–	notOwner, unchangeable, invalidInput, unavailable
get	a: AdultIdT $\cup \{\perp\}$, k: KindT, id: ItemIdT	ItemT	notOwner, unknownItem, unavailable
list	a: AdultIdT $\cup \{\perp\}$, k: KindT, p: LearnerIdT $\cup \{\perp\}$	seq of EntryT	notOwner, unavailable
remove	a: AdultIdT, k: KindT, id: ItemIdT	–	notOwner, unknownItem, unavailable

28.4 Semantics

28.4.1 State Variables

None.

28.4.2 Environment Variables

- *db*: set of tuple of (*kind* : KindT, *adult* : AdultIdT $\cup \{\perp\}$, *learner* : LearnerIdT $\cup \{\perp\}$, *id* : ItemIdT, *item* : ItemT), the saved items in the hosted database. The database's access rules let an adult reach only their own items (NFR-SEC-001, SAF-SHA-001).

28.4.3 Assumptions

- a is the adult that the Adult Access Module (M14) gives as signed in.
- The Attempt Record Module replaces a lesson record only until it ends that record's lesson (FR-REC-009).
- Lessons are stored by the team, outside Squigglr.

28.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `newId` gives an identifier that no item has.

- `put` saves x as item id of kind k, for adult a and learner p, creating it or replacing it. $p = \perp$ is for an item that belongs to the adult rather than to one learner, such as a learner profile. `put` returns only after the server confirms the save (SAF-SHA-003). Repeating a `put` with the same id and x changes nothing, so an attempt that is sent again is still saved once (FR-REC-008).

- A saved attempt record is never replaced: `put` with a different x raises `unchangeable` and logs the refused change with the Audit Log Module, without the record's contents (FR-REC-009, NFR-SEC-003).

- `get` and `list` give only a's items (NFR-SEC-001). `list` gives the items of learner p, or every item of kind k that belongs to a when $p = \perp$. Lessons belong to no adult, so they are read with $a = \perp$.

- `remove` deletes an item. Who may delete records is still an open issue in the SRS (OI-02).

`notOwner` is raised when p or the item is not a's, when $a = \perp$ for a kind other than *lesson*, and when `put` or `remove` is given kind *lesson*. `invalidInput` is raised when the text of x fails the input check (NFR-SEC-006), and `unavailable` when the server does not answer or does not confirm.

28.4.5 Local Functions

None.

29 MIS of the Connection Recovery Module (M24)

29.1 Module

ConnectionRecovery

29.2 Uses

Storage Module (M23).

29.3 Syntax

29.3.1 Exported Constants

None.

29.3.2 Exported Types

- UnsavedT = tuple of (*kind* : KindT, *learner* : LearnerIdT $\cup$ {$\perp$}, *id* : ItemIdT, *item* : ItemT, *kept* : DateTimeT), an item kept on the device that the server has not yet confirmed, and when it was kept. KindT, ItemIdT and ItemT are those of the Storage Module (M23).

29.3.3 Exported Access Programs

Name	In	Out	Exceptions
init	–	–	–
save	*a*: AdultIdT, *k*: KindT, *p*: LearnerIdT $\cup$ {$\perp$}, *id*: ItemIdT, *x*: ItemT	$\mathbb{B}$	notOwner, unchangeable, invalidInput
unsaved	*a*: AdultIdT	seq of UnsavedT	–

29.4 Semantics

29.4.1 State Variables

None. The kept items are in *deviceStore*.

29.4.2 Environment Variables

- *deviceStore*: set of tuple of (*adult* : AdultIdT, *u* : UnsavedT), the items kept in the browser's storage on this device. It stays when the page is closed.
- *connection*: $\mathbb{B}$, whether the browser reports a network connection.

29.4.3 Assumptions

- `init` is called once, when the page is opened, before any other access program.
- a is the adult that the Adult Access Module (M14) gives as signed in.

29.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief:

- `save` keeps the item in *deviceStore* first, then sends it with `put` of the Storage Module. It gives true once the server confirms, and removes the item from *deviceStore*. It gives false if the server does not answer, and the item stays kept (SAF-RAY-003, FR-CRT-025).
- `init`, and every change of *connection* to true, sends each kept item again, until the server confirms it (NFR-PERF-005). Because a repeated `put` changes nothing, an item is never saved twice (FR-REC-008).
- The other exceptions of `put` are passed on, and the item is not kept, because sending it again would fail the same way.
- `unsaved` gives a's kept items, oldest first, so the adult can be told what is not yet saved (SAF-SHA-003).

29.4.5 Local Functions

None.

30 MIS of the Audit Log Module (M25)

30.1 Module

AuditLog

30.2 Uses

None.

30.3 Syntax

30.3.1 Exported Constants

None.

30.3.2 Exported Types

- EventKindT = {*signInFailed*, *sessionTimeout*, *changeRefused*}, the security events that are logged: a failed sign-in and a session timeout (NFR-SEC-005), and a refused change to a saved record (NFR-SEC-003).

30.3.3 Exported Access Programs

Name	In	Out	Exceptions
log	*e*: EventKindT, *a*: AdultIdT ∪ {⊥}, *about*: string	–	–

30.4 Semantics

30.4.1 State Variables

None.

30.4.2 Environment Variables

- *auditLog*: seq of tuple of (*kind* : EventKindT, *adult* : AdultIdT ∪ {⊥}, *about* : string, *at* : DateTimeT), the server-side store of audit events.
- *clock*: DateTimeT, the current time.

30.4.3 Assumptions

- *about* names what the event concerns, such as the email entered or an item's identifier. It never holds a password or the contents of a record (NFR-SEC-003).

30.4.4 Access Routine Semantics

The full semantics are given in Revision 0. In brief, `log` adds the event $(e, a, about, clock)$ to *auditLog*, with $a = \perp$ when no adult is known, as for a failed sign-in with an unknown email. No access program of Squigglr reads *auditLog*, so no adult can see it (NFR-SEC-005).

30.4.5 Local Functions

None.

References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL http://citeseer.ist.psu.edu/428727.html.

Squigglr Team. Module Guide for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/Design/SoftArchitecture/MG.pdf, 2026a.

Squigglr Team. Software Requirements Specification for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/SRS/SRS.pdf, 2026b.

31 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

31.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

31.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

31.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Individual Disclosures

Ahmed: I used Anthropic's Claude Opus 5.5, accessed through Claude Code, while preparing my parts of this document: the introduction, the notation, the types shared between modules and the module decomposition, and the MIS of the Audio Output, Guided Lesson Control, Lesson Catalogue, Attempt, Attempt Evaluation, Feedback, Improvement Measure, Attempt Record and Geometry Modules (M3 to M10, and M20). All the ideas behind these sections are my own, apart from the module list and the uses, which come from the MG, and project decisions were made through discussion with my teammates. I used the model

to turn my notes and decisions into draft wording, and to clean up grammar, sentence structure and clarity, along with LaTeX formatting. I also used it to split my issue into sub-issues and to write clear pull request descriptions, making my changes easier for my teammates to review. I reviewed every AI-assisted passage against the SRS, the MG and the parts of this document my teammates wrote before including it. The final content is my responsibility.

Huzaifah: I used Anthropic's Claude Opus 5.5, accessed through the omp coding agent, for my parts of this document: the MIS of the Adult Access, Learner Profile, Learner Settings, Progress Summary, Adult View, Storage, Connection Recovery and Audit Log Modules (M14 to M18, and M23 to M25). For planning, it read issue #266, the MG and its uses table on issue #263, the SRS, the MIS and Writing checklists and the course's MIS lecture, and helped me split my issue into five sub-issues. It proposed each module's interface in a written plan, built from the secrets in the MG and the SRS requirements each module traces to, and I reviewed and approved that plan before it wrote the sections. When the shared notation and module files of #324 were posted, it matched my sections to them and to the Attempt Record Module's interface. It also drafted my interface notes on issue #263, managed my branch and commits, and opened my pull request. I checked every interface against the uses table on #263, the requirements it traces to and the MIS checklist, and I built the document to confirm that my sections have no errors or undefined references.

32 Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

7. (After you have implemented another team's module, which means this isn't filled in until after the original deadline). What did you learn by implementing another team's module? Were all the details you needed in the documentation, or did you need to make assumptions, or ask the other team questions? If your team also had another team implement one of your modules, what was this experience like? Are there things in your documentation you could have changed to make the process go more smoothly for when an "outsider" completes some of the implementation?

Your reflection should be about concepts learned in your SE education, such as modular decomposition, cohesion, coupling, information hiding, etc. You should critique the quality and utility of the specification you received.

You should create a PR for your implemented module. A link to the PR should be part of your reflection.